

Periodic Queries

Periodic queries are a way to get information out of Prometheus (or Thanos) to be used elsewhere without having to allow access to those services.

The periodic queries are executed on a scheduled pipeline running on ops.GitLab.net from the job `publish-periodic-queries`. The schedule currently runs daily.

The results are stored as job-artifacts and uploaded to GCS bucket in the `gitlab-internal` project, from either of these locations the can be consumed by other services. For example Sisense.

Every time the queries run, a new file is uploaded to the GCS bucket, the filename is prefixed with formatted UTC timestamp which specifies when the queries were run. To know the exact time the sample was taken, we shouldn't rely on this timestamp, but instead look at the values in the file.

For now only instant queries are supported. This means that the results are limited to one sample a day at the time the pipeline schedule runs. If we need higher granularity, we will need to extend this to support range queries.

Format of the result

The results of the queries are stored in a JSON file per topic where each query is a key in the file. The name of the file is the name of the topic, derived from how the query was defined.

For example:

```
{
  "stage_group_error_budget_availability": {
    "success": true,
    "status_code": "200",
    "message": "OK",
    "body": {
      "status": "success",
      "data": {
        "resultType": "vector",
        "result": [
          {
            "metric": {
              "product_stage": "growth",
              "stage_group": "activation"
            },
            "value": [
              1622213770.168,
              "0.9567375328296198"
            ]
          }
        ]
      }
    }
  }
}
```

```

    },
    {
      "metric": {
        "product_stage": "manage",
        "stage_group": "import"
      },
      "value": [
        1622213770.168,
        "0.9967654240024224"
      ]
    },
    < ... snip ... >

```

For each query, there are 4 fields:

1. **success** (Boolean): **true** if we consider the query a success. Which is based on the request's status code and the content of the body. If the status was different from 200 or the body contained an error, this field will contain **false**.
2. **status** (String): The status code of the request to prometheus
3. **message** (String): The message associated with the status
4. **body** (Object): If **success** is **true** this contains the unaltered response from Prometheus. If **success** is false, it could contain the error from the server, or the error parsing the result. Read more about the response format in the Prometheus API documentation

Defining or adjusting queries

The query definitions live in the **periodic-queries** directory in the runbooks repository in jsonnet files per topic. The filename of the files to be included in the result needs to end in **.queries.jsonnet**. The name without the suffix will be used as the topic name. The the resulting file will have a name in the following format: **YYYY-mm-dd_HH:MM:SS/<topic name>.json**.

A topic file could have the following format:

```

local periodicQuery = import './periodic-query.libsonnet';

{
  <query_name>: periodicQuery.new({
    query: <PromQL query>
  })
}

```

This would result in an instant query, without any extra parameters. All other parameters defined in the Prometheus API documention can also be passed here. For now, only **instant** queries are supported.

To validate if the query compiles, we can use the following script:

```
bundle exec scripts/perform-periodic-queries.rb --dry-run -f
periodic-queries/<topic name>.queries.jsonnet
```

The `--dry-run` option prevents actually trying to perform the queries. In normal circumstances the workstation does not have direct access to the Thanos instance we use for querying. To test querying, please read how to connect to thanos from your local machine.

When the new queries are ready, please create a merge request in the runbooks repository. In the merge request, link the queries that will be executed in Thanos so the reviewer can validate the performance of the queries.

All information about using the script is available in its output:

```
bundle exec scripts/perform-periodic-queries.rb --help
```

Connecting to Mimir

Our Mimir instance isn't accessible from the outside. But it can be reached through an SSH tunnel if you have access to a bastion. Most people should have access to a staging bastion. A tunnel can be created like this, creating a SOCKS proxy on port 18202:

```
ssh -l $(whoami) lb-bastion.gstg.gitlab.com -D 18202
```

This will make Mimir available at <https://mimir-internal.ops.gke.gitlab.net/prometheus>. This is the default configuration `scripts/perform-periodic-queries.rb` will use. A different URL can be configured using `PERIODIC_QUERY_PROMETHEUS_URL`.

The `socksify` gem should be used to provide the SOCKS connectivity – only required locally. For example:

```
bundle exec socksify_ruby localhost 18202 scripts/perform-periodic-queries.rb -n
```

Connecting to Thanos

Our Thanos instance isn't accessible from the outside. But it can be reached through an SSH tunnel if you have access to a bastion. Most people should have access to a staging bastion. A tunnel can be created like this:

```
ssh -L 10902:thanos-query-frontend-internal.ops.gke.gitlab.net:9090 lb-bastion.gstg.gitlab.com
```

This will make thanos available at <http://localhost:10902>. This is the default configuration `scripts/perform-periodic-thanos-queries.rb` will use.

Uploading to Google Cloud Storage

To make the `script/perform-periodic-queries.rb` upload results to GCS. The following environment variables need to be set:

1. `PERIODIC_QUERY_GCP_KEYFILE_PATH`: This is the path to a keyfile that allows authenticating as a service account that has The **Storage Object Admin** role on the bucket we want to use.
2. `PERIODIC_QUERY_GCP_PROJECT`: The GCP project for the bucket.
3. `PERIODIC_QUERY_BUCKET`: The name of the bucket to upload into.

For development purposes there is a Bucket with a service count available. Look for **Runbooks Periodic Queries dev-bucket** in the Engineering Vault in 1Password.